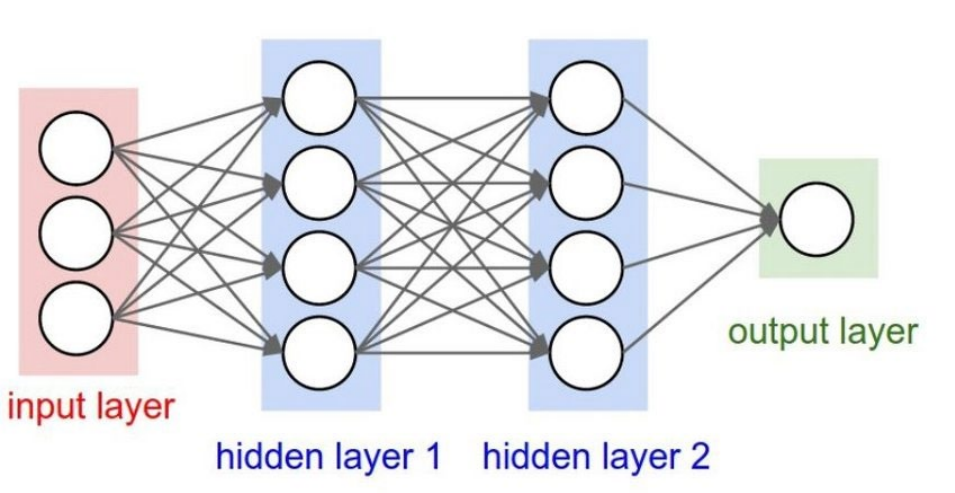
ARTIFICIAL NEURAL NETWORKS: DENSE

Alexander Schoolcraft

Robert Ward

Richard Zheng

WHAT IS A DENSE NEURAL NETWORK?



A **Dense (Fully-Connected) Neural Network** has an input layer, one or more hidden layers, and an output layer. Every unit in one layer connects to every unit in the next (feed-forward). Each layer computes an **affine transformation** followed by a **non-linear activation** ϕ:

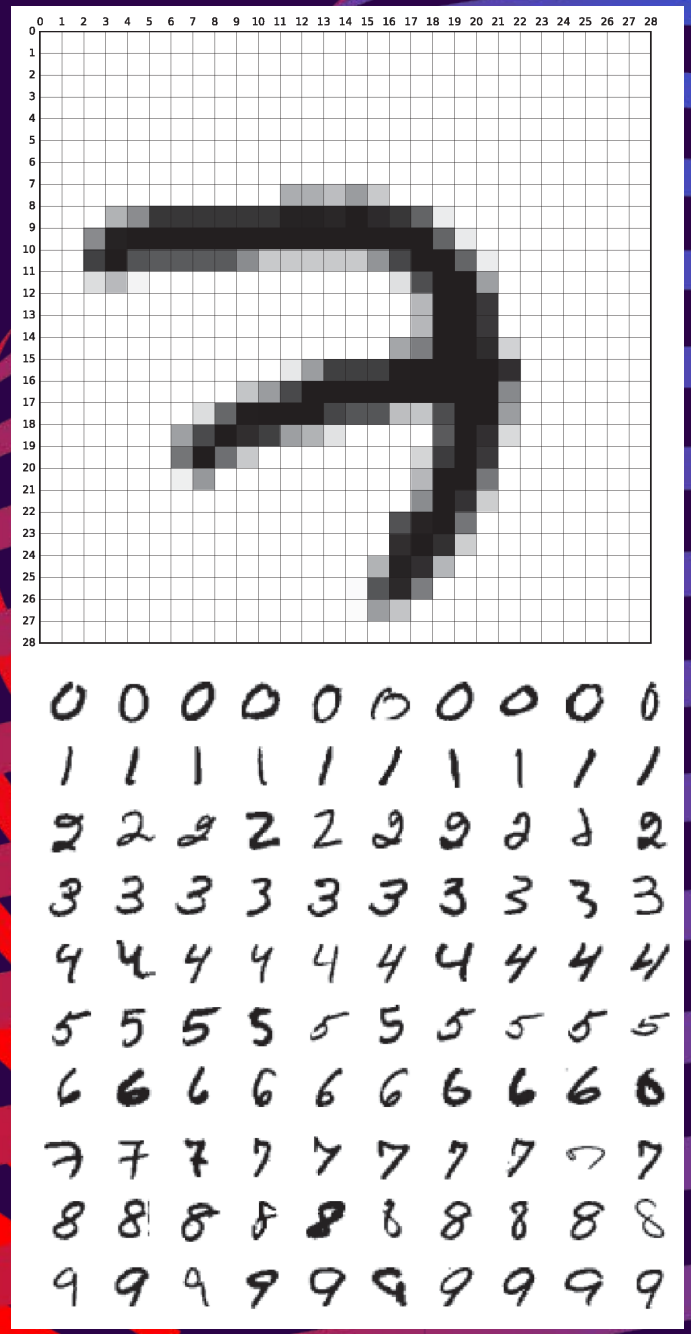
$$z = \sum_{j=1}^{d} x_j \omega_j + \sigma \Rightarrow y = \phi(z)$$

During training, the network learns the weights ω and biases σ by minimizing a loss via gradient descent and backpropagation. In our classifier, hidden layers use ReLU for ϕ. The final layer outputs **logits** for the 10 classes; with **CrossEntropyLoss**, the Softmax is applied **inside the loss**, not as a separate layer.

DATASET

MNIST – 70,000 28x28 pixel greyscale images of hand-written numerals:
{0, 1, 2, 3, 4, 5, 6, 7, 8, 9}

```python
# ---------------------------
# Data: MNIST with normalization
# ---------------------------
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.1307,), (0.3081,))  # mean/std for MNIST
])

data_root = "./data"

full_train = torchvision.datasets.MNIST(
    root=data_root, train=True, download=True, transform=transform
)

test_dataset = torchvision.datasets.MNIST(
    root=data_root, train=False, download=True, transform=transform
)

# Requirement: training 60k, test 10k, plus a validation set of 10k.
# We'll split the original 60k into 50k train / 10k val.
train_size = 50_000
val_size = len(full_train) - train_size  # 10_000
train_dataset, val_dataset = random_split(
    full_train,
    lengths=[train_size, val_size],
    generator=torch.Generator().manual_seed(SEED)
)

print(f"Train: {len(train_dataset)} | Val: {len(val_dataset)} | Test: {len(test_dataset)}")

# Dataloaders
BATCH_SIZE = 128
train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True, num_workers=NUM_WORKERS, pin_memory=True)
val_loader   = DataLoader(val_dataset,   batch_size=BATCH_SIZE, shuffle=False, num_workers=NUM_WORKERS, pin_memory=True)
test_loader  = DataLoader(test_dataset,  batch_size=BATCH_SIZE, shuffle=False, num_workers=NUM_WORKERS, pin_memory=True)
```



MODEL

DENSE/LINEAR (FULLY CONNECTED) ANN

```python
# ---------------------------
# Model: Simple Feedforward ANN (784 -> 256 -> 128 -> 10)
# ---------------------------
class MLP(nn.Module):
    def __init__(self, in_dim=784, h1=256, h2=128, out_dim=10, p_drop=0.1):
        super().__init__()
        self.fc1 = nn.Linear(in_dim, h1)
        self.fc2 = nn.Linear(h1, h2)
        self.fc3 = nn.Linear(h2, out_dim)
        self.dropout = nn.Dropout(p_drop)

    def forward(self, x):
        # x: (B, 1, 28, 28) -> flatten to (B, 784)
        x = x.view(x.size(0), -1)
        x = F.relu(self.fc1(x))
        x = self.dropout(x)
        x = F.relu(self.fc2(x))
        x = self.dropout(x)
        logits = self.fc3(x)            # raw scores (logits)
        return logits                    # softmax handled by CrossEntropyLoss internally

model = MLP().to(device)
print(model)

# ---------------------------
# Loss, Optimizer, Scheduler
# ---------------------------
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=1e-3, weight_decay=1e-4)
scheduler = optim.lr_scheduler.StepLR(optimizer, step_size=10, gamma=0.5)
```

Input layer:

- 784 input nodes (28*28)

Hidden layers:

- 256 nodes, ReLU activation, 10% dropout
- 128 nodes, ReLU activation, 10% dropout

Output layer:

- 10 output nodes, Cross-Entropy loss (applies LogSoftmax internally)

Optimization: Adam

Scheduler: StepLR

SUBMITTING THE BATCH JOB

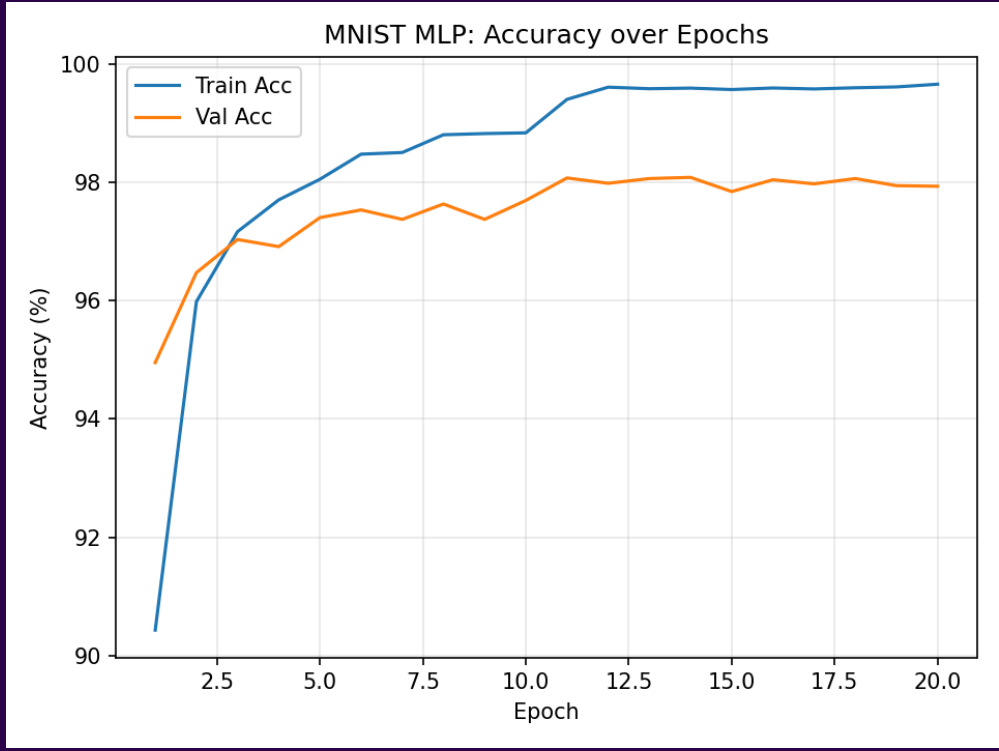
```
(myEnv) [schoolcr@bridges2-login011 DNN]$ cat neural_slurm.sh
#!/bin/bash
#SBATCH -J nn_short             # Job name
#SBATCH -A cis250151p           # PCS allotment name
#SBATCH -p GPU-shared           # Partition (queue)
#SBATCH --gres=gpu:v100-16:1    # 1 V100 16GB GPU on a shared node
#SBATCH -t 00:10:00             # Walltime HH:MM:SS (<= 10 minutes)
#SBATCH -N 1                    # One node
#SBATCH --ntasks=1              # One task (single process)
#SBATCH --cpus-per-task=4       # Some CPU cores to feed the GPU
#SBATCH --mem=16G               # Host RAM
#SBATCH -o neural_net_out.txt   # Stdout
#SBATCH --mail-user=schoolcr    # Email job updates
#SBATCH --mail-type=ALL         # Email all updates



# ----- Safety / setup -----
set -euo pipefail
# Start in the correct directory
cd /ocean/projects/cis250151p/schoolcr/DNN

# ----- Environment -----
# Use PSC's Anaconda
module purge
module load anaconda3

# Activate Conda environment for job with all reqired modules installed
conda activate neural || true

# Confirm devices visible to the job
nvidia-smi || true
python --version

# ----- Run script -----
python NeuralNets.py
```

MODEL TRAINING

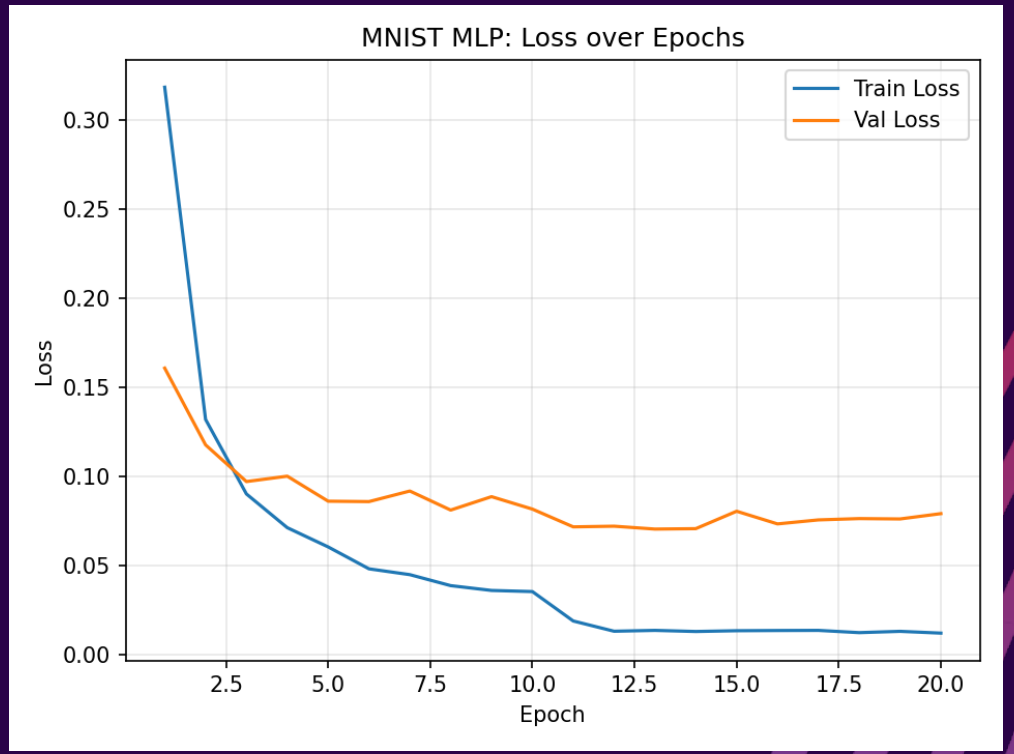
```python
# ---------------------------
# Train loop
# ---------------------------
EPOCHS = 20
history = {
    "train_loss": [],
    "train_acc":  [],
    "val_loss":   [],
    "val_acc":    [],
}

for epoch in range(1, EPOCHS + 1):
    train_loss, train_acc = run_epoch(train_loader, train_mode=True)
    val_loss,   val_acc   = run_epoch(val_loader,   train_mode=False)

    history["train_loss"].append(train_loss)
    history["train_acc"].append(train_acc)
    history["val_loss"].append(val_loss)
    history["val_acc"].append(val_acc)

    scheduler.step()

    print(f"Epoch {epoch:02d}/{EPOCHS} | "
          f"Train Loss: {train_loss:.4f}  Acc: {train_acc*100:5.2f}% | "
          f"Val Loss: {val_loss:.4f}  Acc: {val_acc*100:5.2f}%")
```

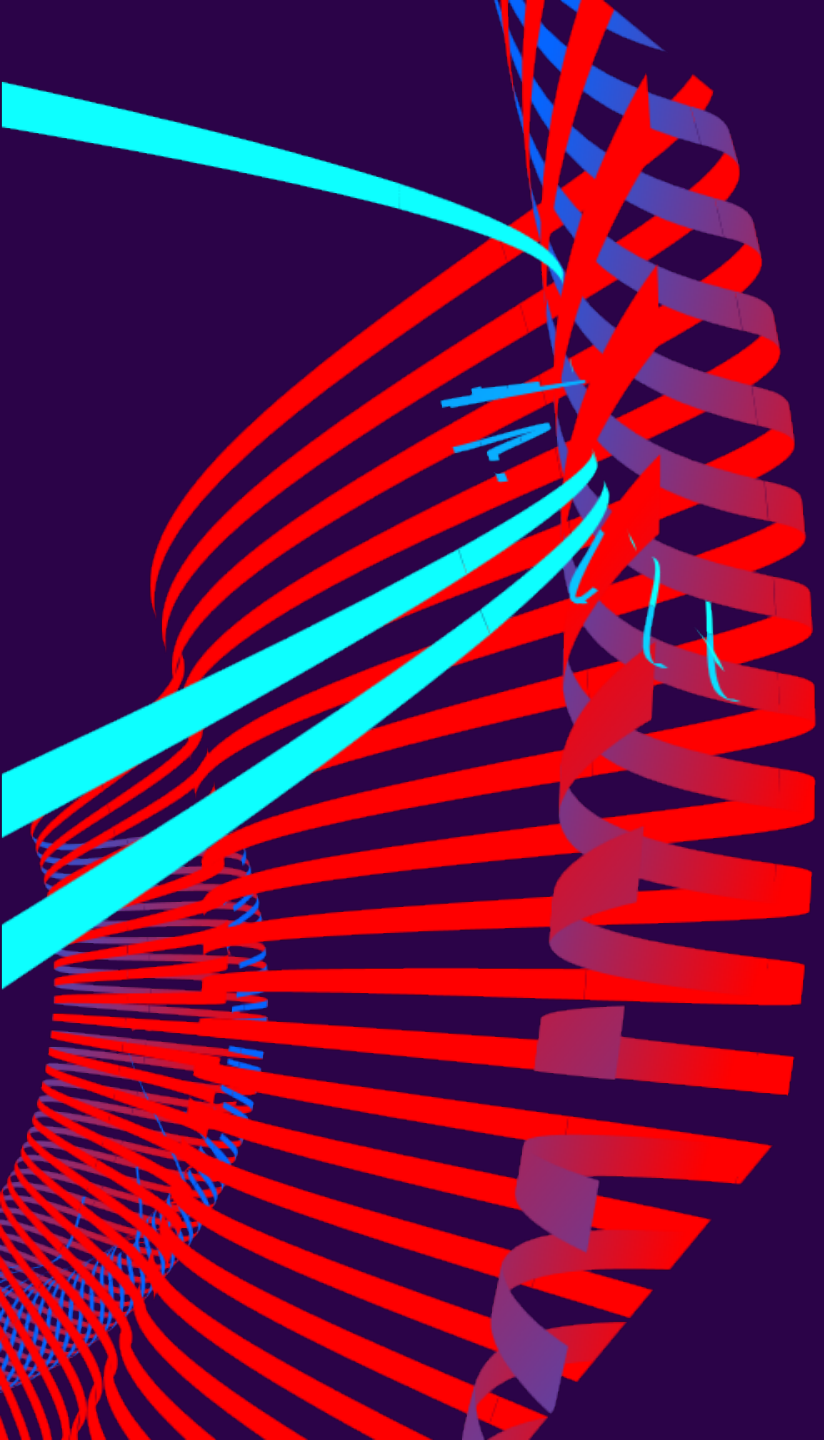
```
Train: 50000 | Val: 10000 | Test: 10000
MLP(
  (fc1): Linear(in_features=784, out_features=256, bias=True)
  (fc2): Linear(in_features=256, out_features=128, bias=True)
  (fc3): Linear(in_features=128, out_features=10, bias=True)
  (dropout): Dropout(p=0.1, inplace=False)
)
Epoch 01/20 | Train Loss: 0.3182  Acc: 90.43% | Val Loss: 0.1606  Acc: 94.95%
Epoch 02/20 | Train Loss: 0.1317  Acc: 95.98% | Val Loss: 0.1175  Acc: 96.47%
Epoch 03/20 | Train Loss: 0.0899  Acc: 97.16% | Val Loss: 0.0969  Acc: 97.03%
Epoch 04/20 | Train Loss: 0.0710  Acc: 97.70% | Val Loss: 0.0999  Acc: 96.91%
Epoch 05/20 | Train Loss: 0.0602  Acc: 98.05% | Val Loss: 0.0859  Acc: 97.40%
Epoch 06/20 | Train Loss: 0.0479  Acc: 98.47% | Val Loss: 0.0856  Acc: 97.53%
Epoch 07/20 | Train Loss: 0.0446  Acc: 98.50% | Val Loss: 0.0916  Acc: 97.37%
Epoch 08/20 | Train Loss: 0.0385  Acc: 98.80% | Val Loss: 0.0809  Acc: 97.63%
Epoch 09/20 | Train Loss: 0.0358  Acc: 98.82% | Val Loss: 0.0884  Acc: 97.37%
Epoch 10/20 | Train Loss: 0.0351  Acc: 98.83% | Val Loss: 0.0814  Acc: 97.69%
Epoch 11/20 | Train Loss: 0.0186  Acc: 99.40% | Val Loss: 0.0715  Acc: 98.07%
Epoch 12/20 | Train Loss: 0.0129  Acc: 99.60% | Val Loss: 0.0718  Acc: 97.98%
Epoch 13/20 | Train Loss: 0.0134  Acc: 99.58% | Val Loss: 0.0702  Acc: 98.06%
Epoch 14/20 | Train Loss: 0.0127  Acc: 99.59% | Val Loss: 0.0704  Acc: 98.08%
Epoch 15/20 | Train Loss: 0.0132  Acc: 99.56% | Val Loss: 0.0802  Acc: 97.84%
Epoch 16/20 | Train Loss: 0.0133  Acc: 99.59% | Val Loss: 0.0731  Acc: 98.04%
Epoch 17/20 | Train Loss: 0.0133  Acc: 99.57% | Val Loss: 0.0754  Acc: 97.97%
Epoch 18/20 | Train Loss: 0.0121  Acc: 99.59% | Val Loss: 0.0761  Acc: 98.06%
Epoch 19/20 | Train Loss: 0.0128  Acc: 99.61% | Val Loss: 0.0759  Acc: 97.94%
Epoch 20/20 | Train Loss: 0.0118  Acc: 99.65% | Val Loss: 0.0789  Acc: 97.93%

Test Loss: 0.0694 | Test Acc: 98.22%
Saved plots to figs/loss_curves.png and figs/accuracy_curves.png
```

MODEL TRAINING



Over the 20 training epochs, towards the end, we started to see some minor overfitting, as evidenced by the loss starting to bounce around ~0.012. This indicates a minor overfit over the last 2-3 epochs, however, it was not significant and allowed for slightly increased accuracy.

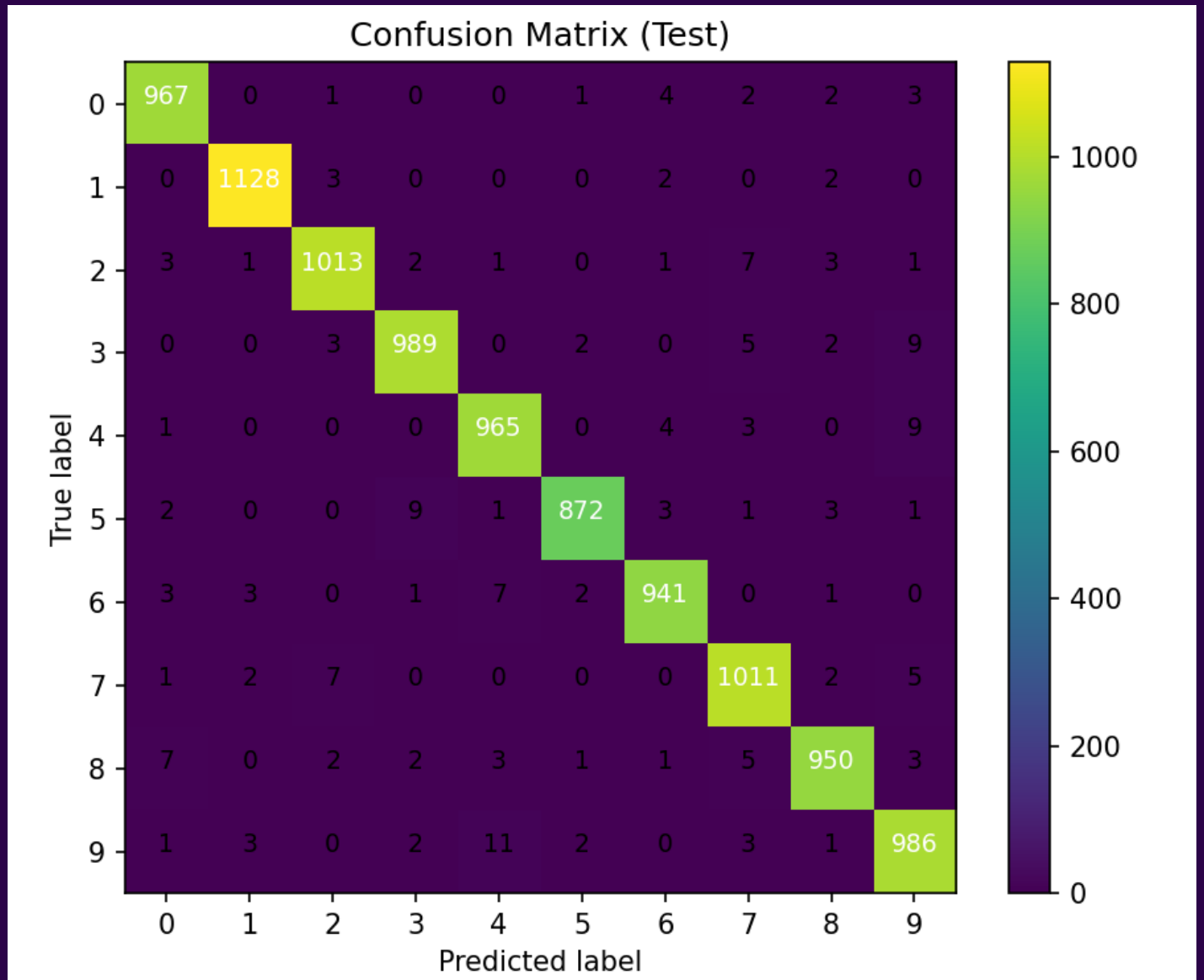




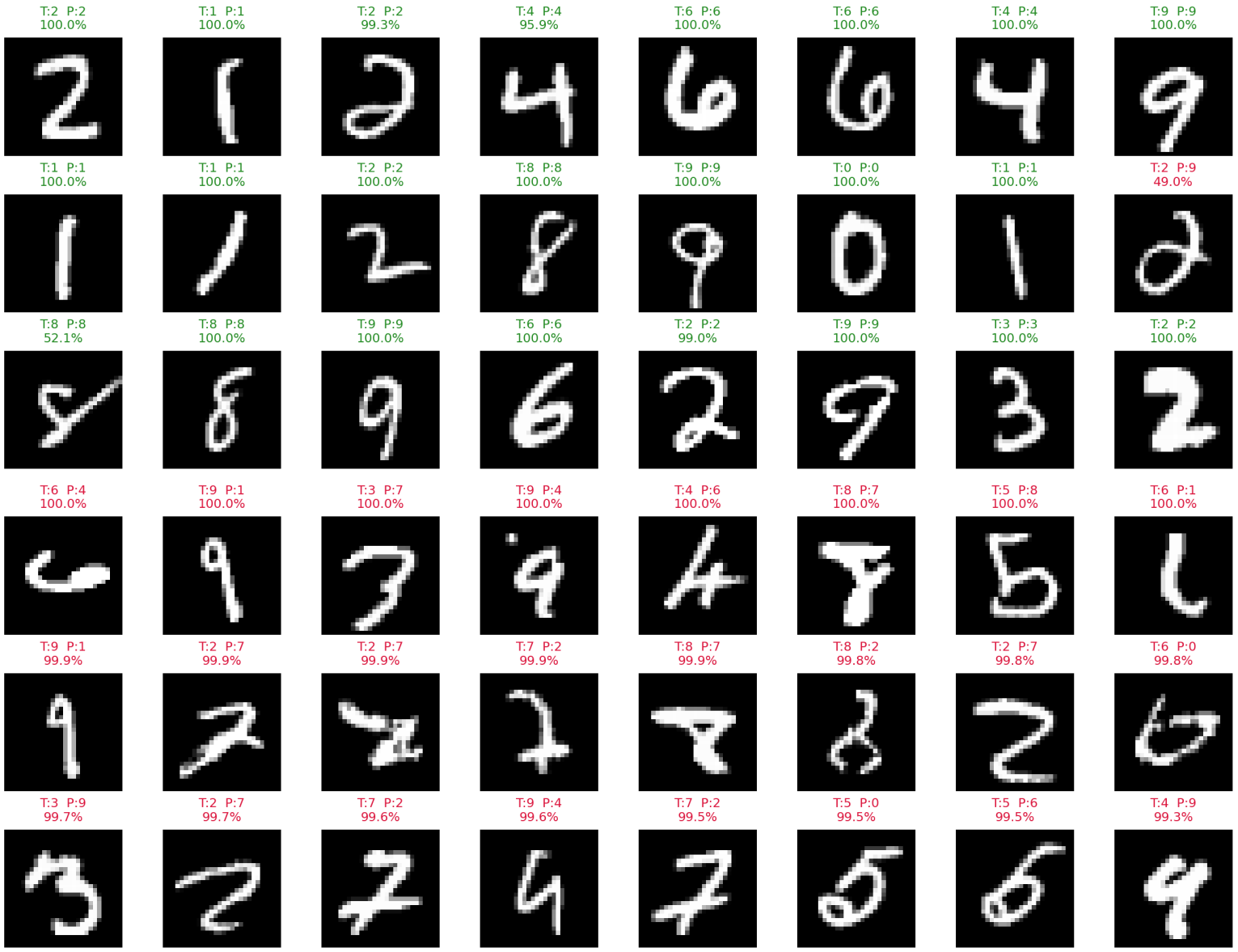
RESULTS

```
Classification Report:
              precision    recall  f1-score   support

           0     0.9817    0.9867    0.9842       980
           1     0.9921    0.9938    0.9930      1135
           2     0.9845    0.9816    0.9830      1032
           3     0.9841    0.9792    0.9816      1010
           4     0.9767    0.9827    0.9797       982
           5     0.9909    0.9776    0.9842       892
           6     0.9843    0.9823    0.9833       958
           7     0.9749    0.9835    0.9792      1028
           8     0.9834    0.9754    0.9794       974
           9     0.9695    0.9772    0.9733      1009

    accuracy                         0.9822     10000
   macro avg     0.9822    0.9820    0.9821     10000
weighted avg     0.9822    0.9822    0.9822     10000

Saved confusion matrix to figs/confusion_matrix.png
Saved random predictions grid â†’ figs/test_random_preds.png
Saved top misclassifications grid â†’ figs/test_top_miscls.png
Saved sample predictions table â†’ figs/sample_preds.csv
```

RESULTS

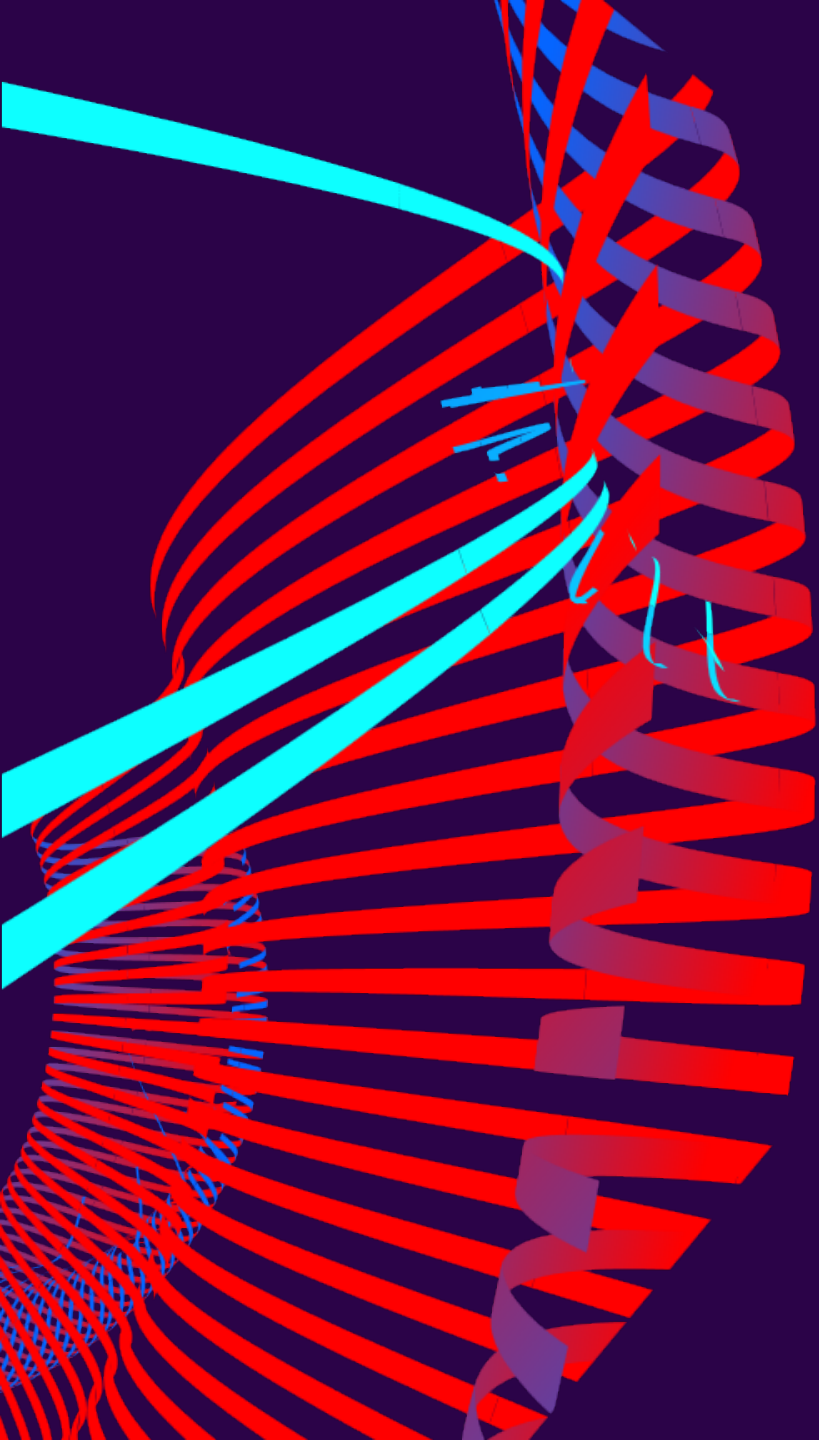

Confusion Matrix (Test)
True label
Predicted label
0 967 0 1 0 0 1 4 2 2 3
1 0 1128 3 0 0 0 2 0 2 0
2 3 1 1013 2 1 0 1 7 3 1
3 0 0 3 989 0 2 0 5 2 9
4 1 0 0 0 965 0 4 3 0 9
5 2 0 0 9 1 872 3 1 3 1
6 3 3 0 1 7 2 941 0 1 0
7 1 2 7 0 0 0 0 1011 2 5
8 7 0 2 2 3 1 1 5 950 3
9 1 3 0 2 11 2 0 3 1 986
0 1 2 3 4 5 6 7 8 9
1000
800
600
400
200
0

RESULTS



RANDOM SAMPLES

TOP MISCLASSIFICATIONS



RESULTS

index	TRUE	pred	confidence
0	7	7	0.9999669790267944
1	2	2	0.9999994039535522
2	1	1	0.9999862909317017
3	0	0	0.9999880790710449
4	4	4	0.9999698400497437
5	1	1	0.9999877214431763
6	4	4	0.9995545744895935
7	9	9	0.9999381303787231
8	5	5	0.7989499568939209
9	9	9	0.9999740123748779
10	0	0	1
11	6	6	0.9999352693557739
12	9	9	0.9999970197677612
13	0	0	1
14	1	1	1
15	5	5	0.9999271631240845
16	9	9	0.9999057054519653
17	7	7	0.9999992847442627
18	3	3	0.9982153177261353
19	4	4	0.9999992847442627
20	9	9	0.9999545812606812
21	6	6	0.9999898672103882
22	6	6	0.999955416
23	5	5	1

CONTRIBUTIONS:

ALEXANDER	Dataset import Model Construction
ROBERT	Training Loops Output Generation
RICHARD	Model Testing Documentation